CHAPTER 3

SOFTWARE LIFE-CYCLE MODELS

Overview

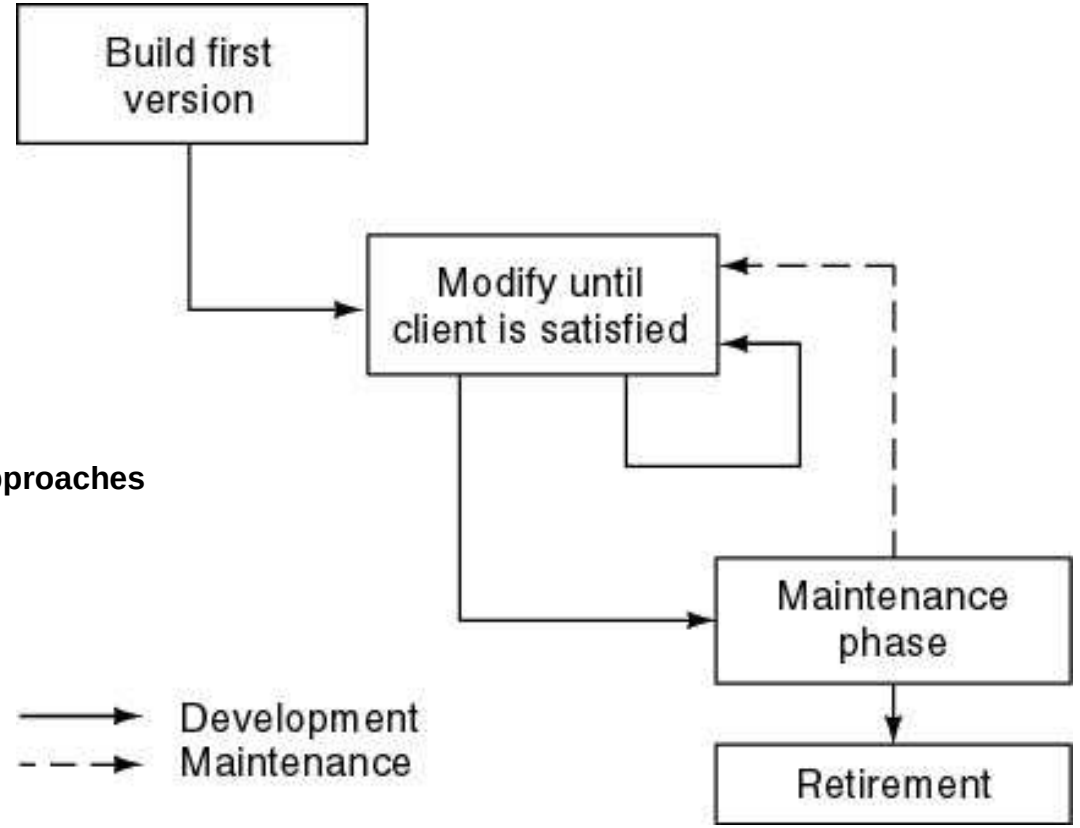
- Build-and-fix model
- Waterfall model
- Rapid prototyping model
- Incremental model
- Extreme programming
- Synchronize-and-stabilize model
- Spiral model
- Object-oriented life-cycle models
- Comparison of life-cycle models

Software Life-Cycle Models

- Life-cycle model
- The steps through which the product progresses
  - Requirements phase
  - Specification phase
  - Design phase
  - Implementation phase
  - Integration phase
  - Maintenance phase
  - Retirement

Build and Fix Model

The Build and Fix Model is an unplanned approach where software is developed and repeatedly modified without formal requirements, design, or structured phases



- One of the **earliest software development approaches**
- Developers **start coding immediately**
- Software is **built first, then errors are fixed**
- **No proper requirement analysis**
- **No system design or planning**
- Changes are made **after problems appear**
- Development is **unstructured**
- Works only for **very small projects**
- Leads to **messy and unorganized code**
- Difficult to **maintain and upgrade**
- High chance of **bugs and failures**
- No defined **phases** (analysis, design, testing, etc.)
- No **milestones or documentation**
- Not suitable for **large or complex systems**
- Replaced by **life-cycle models** like Waterfall and Agile

- **Example: Small Shop Owner's Billing Software**
- A local shop owner wants simple billing software.
- Instead of proper planning:
- A programmer quickly builds a basic system
  → Just to print bills.
- After using it, the owner says:
- "Add GST calculation."
- Programmer **fixes** the softw
- Then owner says:
- "Add stock management."
- Again, programmer modifies the same program.
- Later:
- "Add customer database."
  "Add discount option."
  "Add barcode scanning."
- Each time, code is **changed without proper design**.

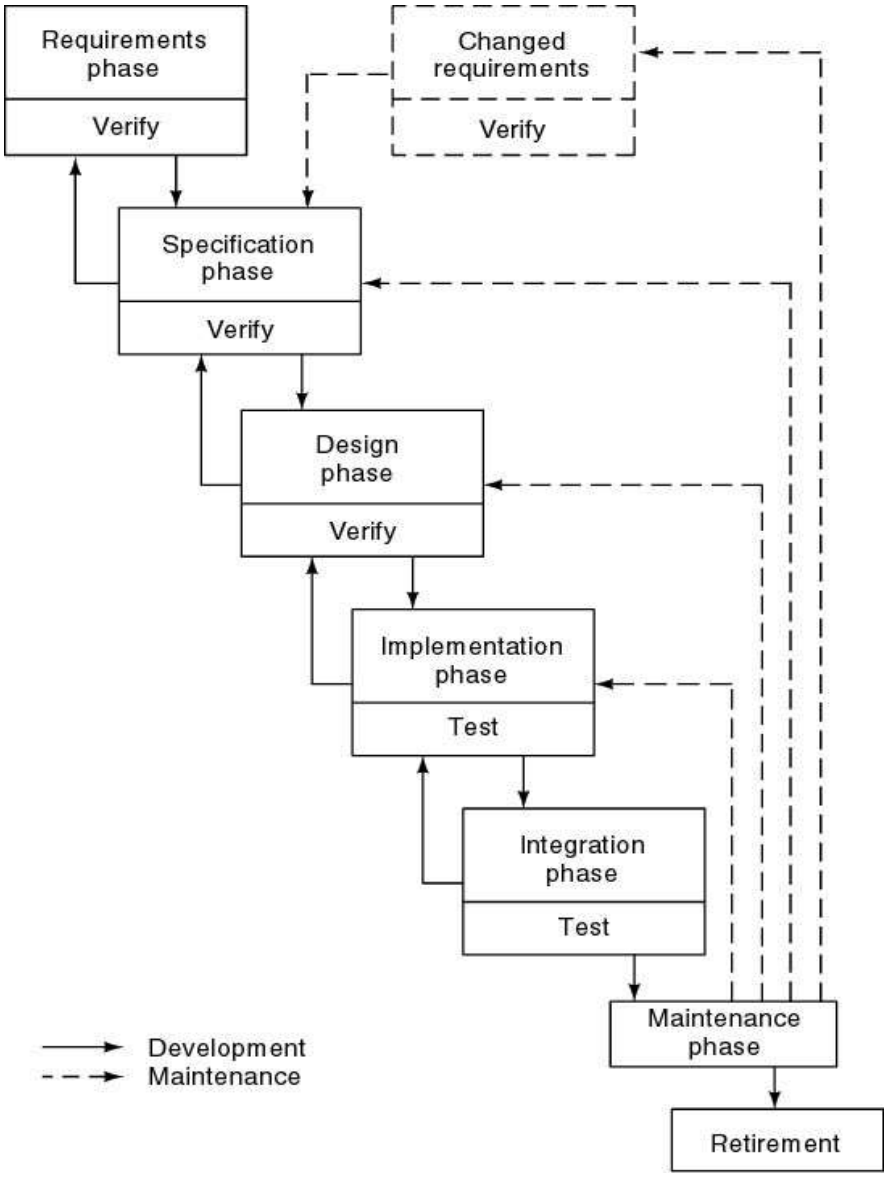
A small shop's billing software that is built quickly and repeatedly modified to fix errors and add features without proper planning

Waterfall Model

- The **Waterfall Model** is a traditional software development model used widely until the early 1980s.
  Development is done **step by step**, and each phase must be completed before moving to the next.

Characteristics

- Follows a **linear sequence** of phases
- **Documentation-driven** approach
- Each phase requires **SQA approval**
- Limited **feedback loops**
- **Advantages**
- Well-structured and **disciplined**
- Proper **documentation**
- **Easy maintenance** due to clear records
- **Disadvantages**
- Requirements are **hard for clients to understand**
- Changes are **difficult and costly**
- Errors are found **late**



Examples

1. Building a House 🏠

- Requirements (design, rooms, budget) are fixed first
- Construction starts only after approval
- Changes after construction are **costly and difficult**
  ➡ Matches the **Waterfall Model**

2. Tailoring a Suit 👔

- Measurements and design are finalized once
- Suit is stitched step by step
- Customer sees the final suit only at the end
  ➡ Little feedback, hard to change later

3. Printed Textbook 📚

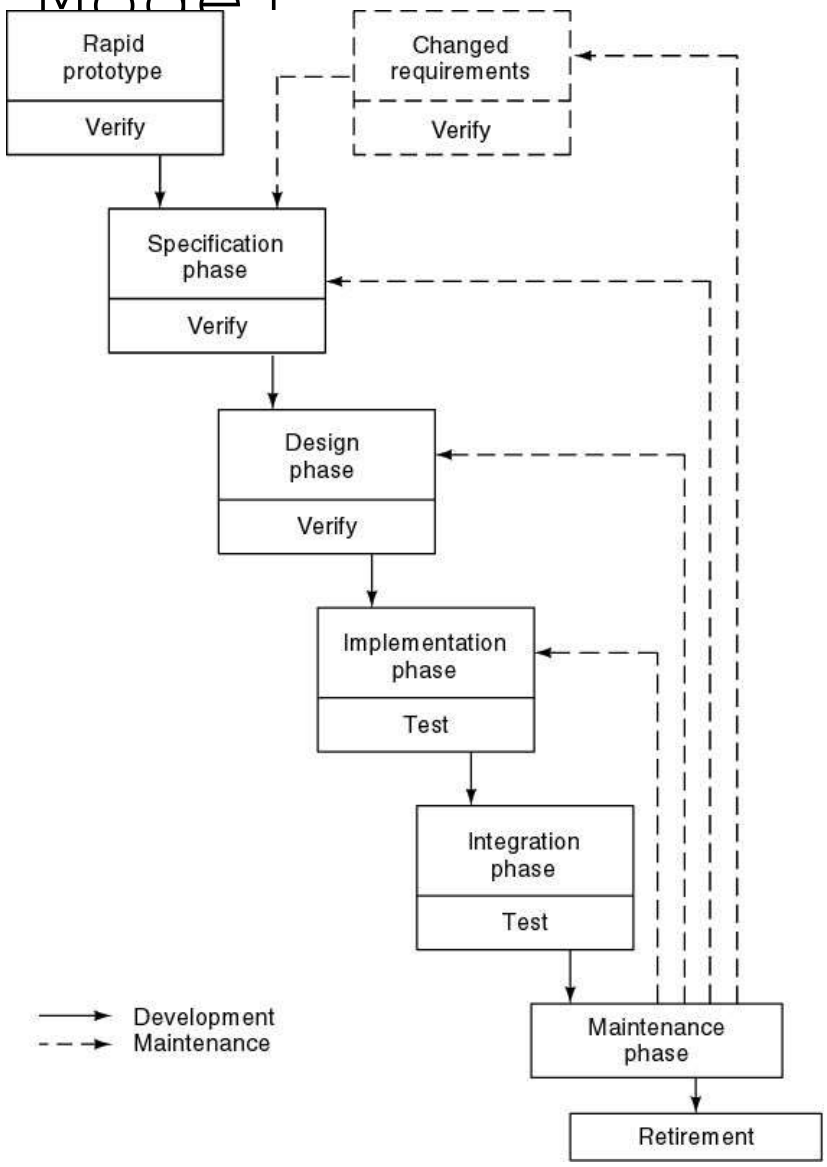
- Content is written → reviewed → printed
- Once printed, changes are **almost impossible**
  ➡ Follows a strict sequence like Waterfall

Rapid Prototyping Model

The **Rapid Prototyping Model** focuses on quickly creating a **working model (prototype)** of the software to understand user requirements.

Key Points

- A **rapid prototype** is a **partial working version** of the final product
- Built to understand **what the client really needs**
- Clients and users **try the prototype**
- Once they are **satisfied,** development moves to the next phase
- Final specifications are taken from the **approved prototype**



Example

- **Demo mobile app** shown to a client before full development
- Client tests screens and flow, gives approval
- Final app is built based on that prototype

Key Points

Important Points (Rapid Prototyping)

- **Do not convert the prototype into the final product**
- Prototype is only for **understanding requirements**
- Rapid prototyping can **replace the specification phase**
- It should **never replace the design phase**

Comparison

Waterfall Model

- Try to get everything **right the first time**
- Requirements are fixed early

Rapid Prototyping Model

- **Frequent changes** based on client feedback
- When client is satisfied, **discard the prototype**
- Then start actual development

Integrating Waterfall and Rapid Prototyping Models

Waterfall Model

- Has **many successful projects**
- But sometimes **client needs are not fully met**

Rapid Prototyping Model

- Has **some success**
- But **not fully proven**
- Has its **own problems**

Best Solution

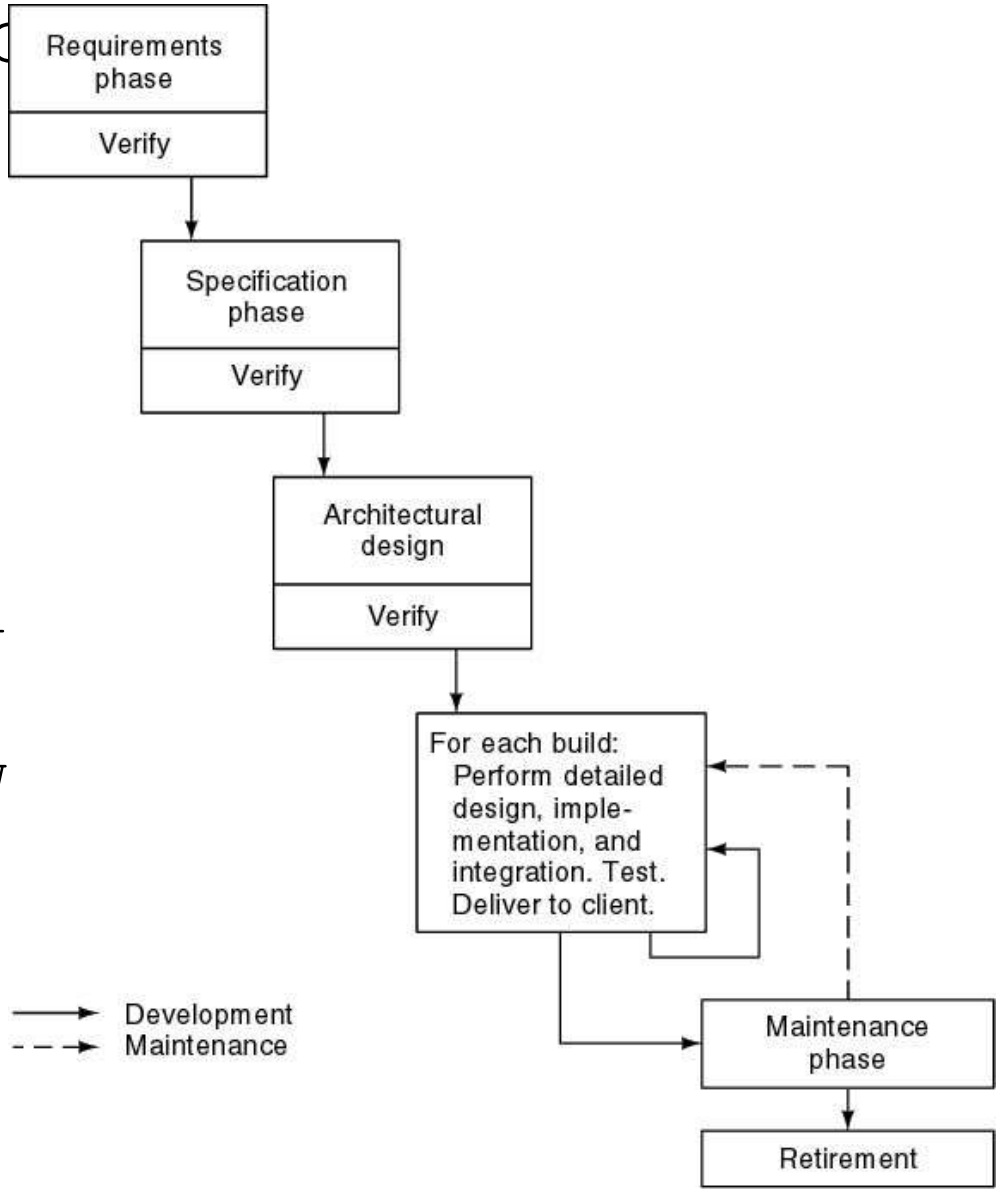
- Use **Rapid Prototyping** to **understand requirements**
- Use **Waterfall Model** for the **remaining phases** (design, coding, testing, maintenance)

Incremental Mod

- In the **Incremental Model**, the software is developed and delivered in **small parts (increments).**

Key Points

- Product is built as a **series of builds**
- Each **build** adds a new **functionality**
- Every build is **designed, coded, tested, and integrated**
- Working software is



Advantages (Pros)

- Working software available **early**
- Easy to **add new features**
- Errors are found **early**
- Client feedback can be taken after each build

Disadvantages (Cons)

- Needs **good planning and design**
- Integration can be **complex**
- Too many builds may cause **inefficiency**
- Not suitable if system is **highly interdependent**

Example

- A **mobile app** released first with login
- Later updates add chat, payments, notifications

Incremental Model (contd)

Waterfall & Rapid Prototyping Models

- Deliver a **fully working product only at the end**
- Users must **wait a long time** to use the software
- Changes late in development can be **stressful (traumatic)**

Incremental Model

- Delivers a **working part of the product early** (within weeks)
- Users start using the system **step by step**
- Development is **less stressful** because changes are smaller
- Requires **less initial investment**
- Gives **faster return on investment (ROI)**

Important Note

- Incremental model needs an **open and flexible**

Example

Online Banking App

- **First release (within weeks):**
  Login + balance check
- **Second release:**
  Money transfer
- **Later releases:**
  Bill payments, statements, offers
- Users get a **working part early**, not the full system at once.

Comparison Example

- **Waterfall / Rapid Prototyping**
- Entire banking system is developed
- Users can use it **only after everything is complete**
- **Incremental Model**
- Users start using **basic features early**
- New features are added **step by step**

Concurrent Incremental Model

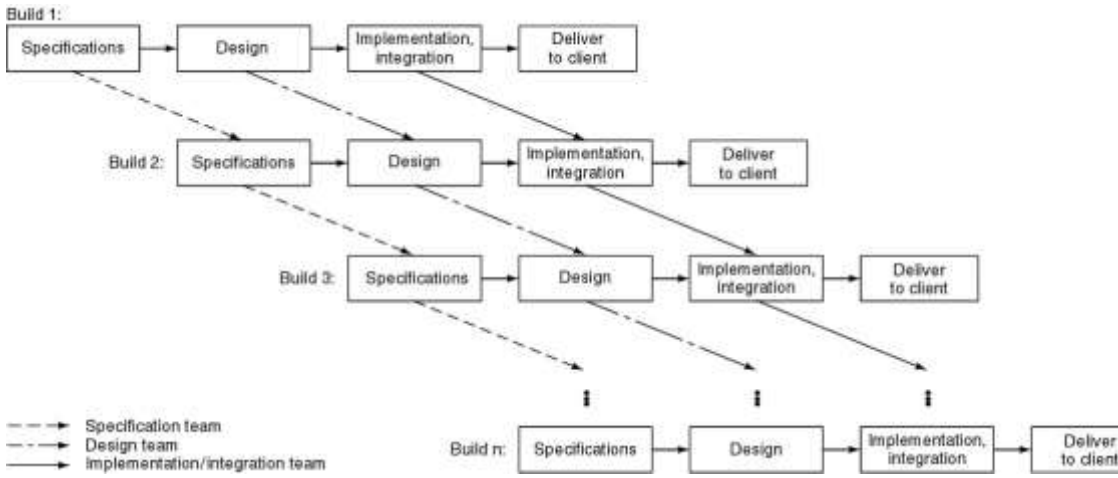
The **Concurrent Incremental Model** develops **multiple increments at the same time** instead of one after another.

Key Points

- Different parts are developed **in parallel**
- **More risky** because pieces may **not fit together properly**
- Requires **strong coordination and planning**
- **CABTAB (Code A Bit And Test A Bit)**
- Developers code a small part and test it immediately
- Sounds fast, but can be **dangerous**

Dangers of CABTAB

- Poor overall design
- Integration problems later
- System becomes hard to maintain



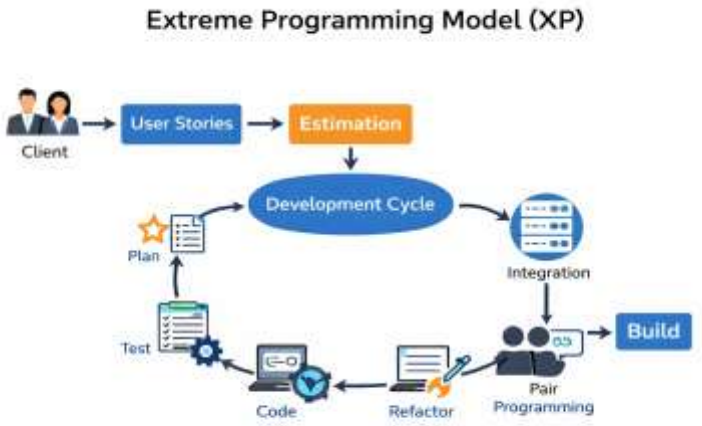
Example

- Multiple teams build **login**, **payment**, **and reports** at the same time
- If not well designed, these parts may **clash during integration**

Extreme Programming

- Work is done in **small parts (increments)**.
- Client gives **user stories (features they want)**.
- Team estimates **time and cost** for each story.
- Stories are selected for the next build.
- Work is divided into **small tasks**.
- **Tests are written first** befo coding.
- Two programmers work togethe (**pair programming**).
- Code is added and tested regularly (**continuous integration**).

Goal:



Unusual Features of XP

- Team works together in one open room.
- A client representative is always available.
- Team should not work overtime for many weeks in a row.
- No fixed roles — everyone shares responsibilities.
- All members help in requirements, design, coding, and testing.
- Code is improved regularly (refactoring).
- No complete design at the start.
- Design changes and improves during development.

Main idea: Teamwork, flexibility, and continuous improvement.

Evaluating XP

- XP has been successful in some projects.
- It works well when requirements are unclear or frequently changing.
- It is still early to fully judge its long-term effectiveness.

Conclusion: XP is useful for flexible projects, but its overall impact is still being evaluated.

Example: Online Food Delivery App

- A company is developing a food delivery app.
- Customers keep suggesting new features (live tracking, wallet payment, offers).
- Requirements change frequently.
- The team works in small releases (adding one feature at a time).
- Tests are written before coding.
- Two developers work together (pair programming).
- The client representative gives regular feedback.
- The app is updated frequently with improvements.

Why XP fits here?
Because the requirements keep changing, and quick updates with continuous feedback are

Synchronize and Stabilize Model

- Developed by **Microsoft**.
- Based on the **incremental model**.
- Requirements are collected by talking to customers.
- Specifications are prepared.
- Project is divided into **3 or 4 builds**.
- Small teams work in parallel on different parts.

Additional Points:

- At the end of each day → **Synchronize** (test and fix errors).
- At the end of each build → **Stabilize** (freeze the build after testing).
- All components are regularly checked to ensure they work together.
- Helps get early feedback about how the product works.

- **Main Idea**:
  Develop in parts, test daily, and keep the system stable after each build.

Mobile Banking App Update

A bank is developing a **new version of its mobile banking app.**

Step 1: Requirements

- Bank collects feedback from customers (need UPI improvement, loan section, better security).
- Specifications are prepared.

Step 2: Divide into Builds

- Project is divided into 3 builds:
- **Build 1** – New user interface
- **Build 2** – Payment & UPI features
- **Build 3** – Security & notifications
- Small teams work on each build at the same time.

During Development:

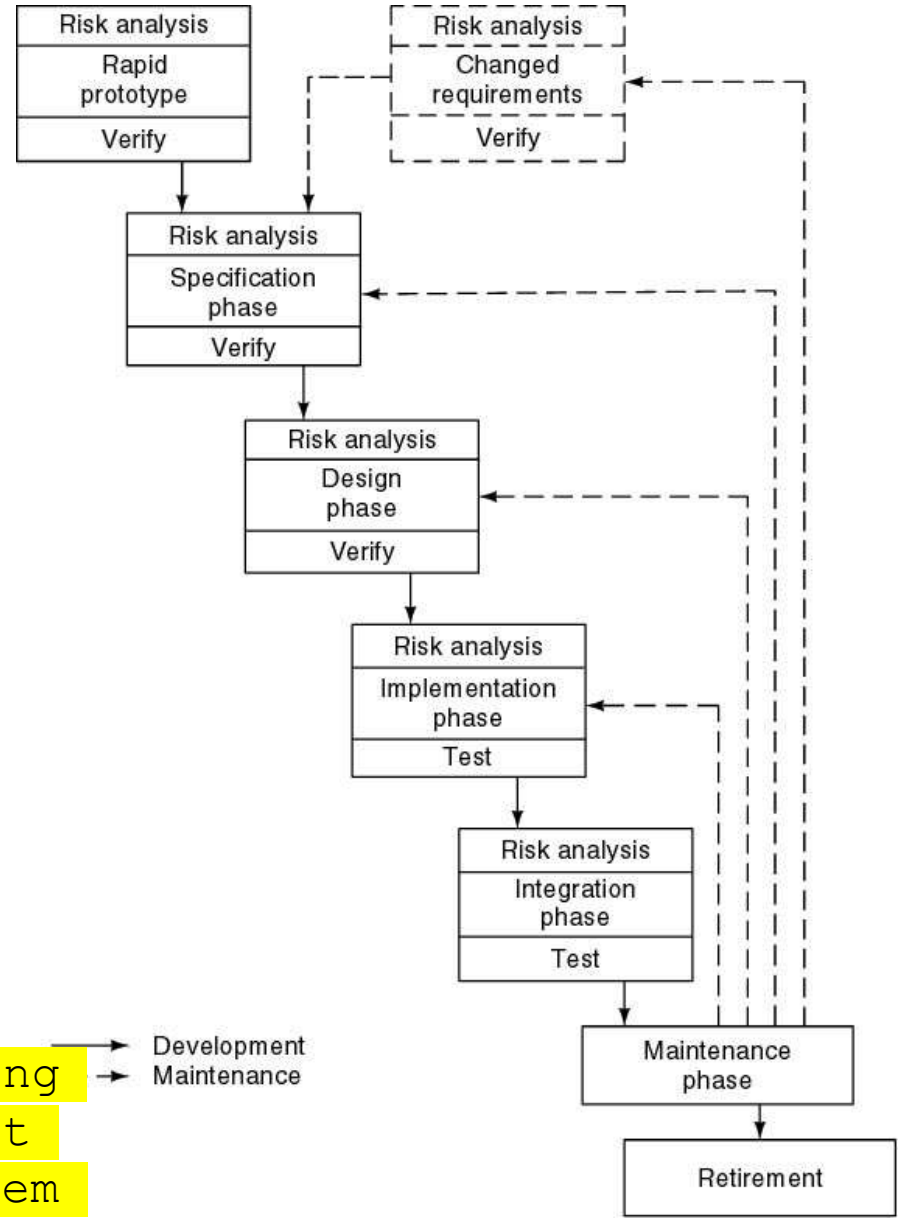
- Every day → All code is combined and tested (**synchronize**).
- Bugs are fixed daily.
- After each build → The version is tested completely and frozen (**stabilize**).

Why this model fits?

- Different teams can work in parallel.
- Problems are found early.
- Final app version is stable and reliable before release.
- **Main Idea:** Work in parts, test daily, and release a stable version step by step.

Spiral Model

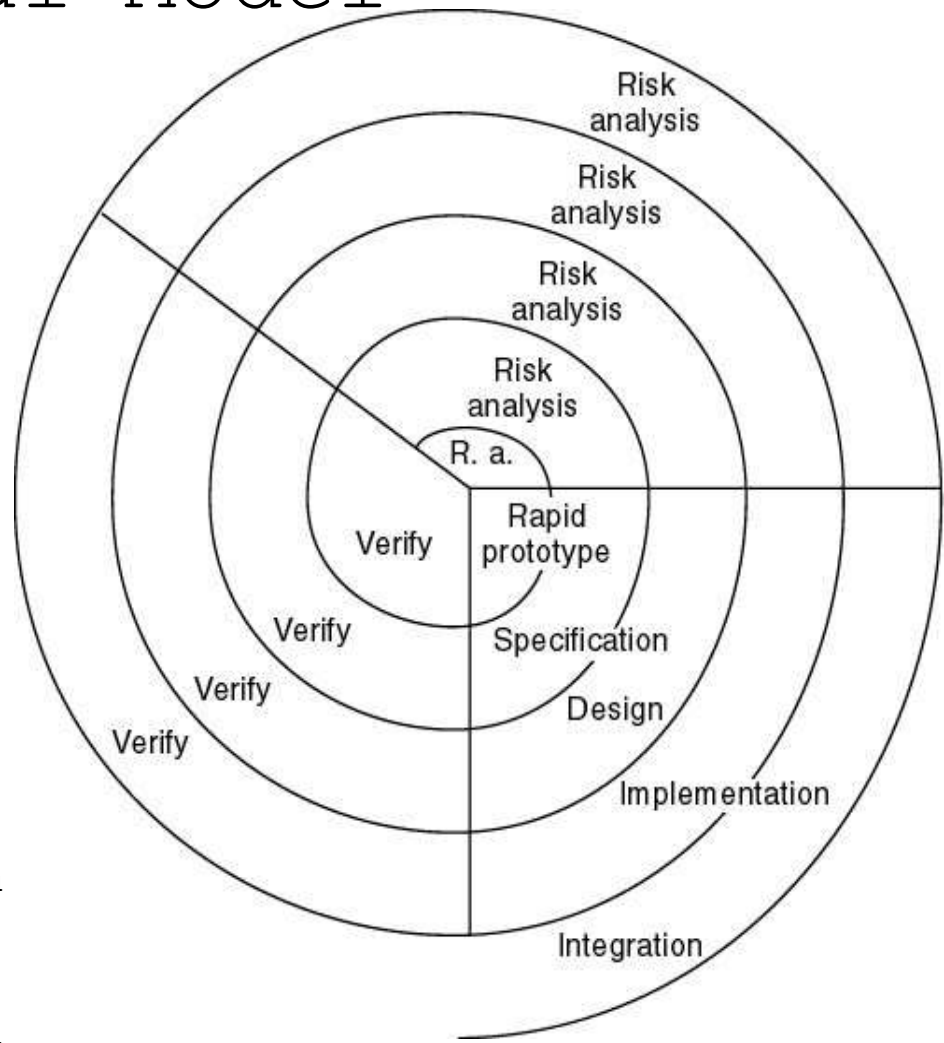
- It is a **Waterfall model with risk analysis.**
- Uses **prototypes** to test ideas early.
- **Before each phase:**
- Consider different **alternatives.**
- Do **risk analysis** (identify possible problems).
- **After each phase:**
- Do **evaluation** (check results).
- Plan the **next phase.**
- **Main Idea:**
  Develop step by step in



Risk analysis means identifying possible problems in a project and planning how to handle them before they happen.

Simplified Spiral Model

- Think of it as a project that moves in circles (loops). Every time you complete a loop, you've made the project better, but only after you've checked for **risks** that could break it.
- If a risk cannot be fixed, the project is killed immediately.



4 Major Risks to Watch For

According to your notes, these are the "project killers" you must monitor during every cycle:

1. **Timing Constraints:** Is the deadline impossible?
2. **Lack of Personnel:** Do we have enough people to do the work?
3. **Competence of Team:** Does the team actually have the skills to build this?
4. **Hardware Dependency:** Are we stuck waiting for a physical part that might never arrive?

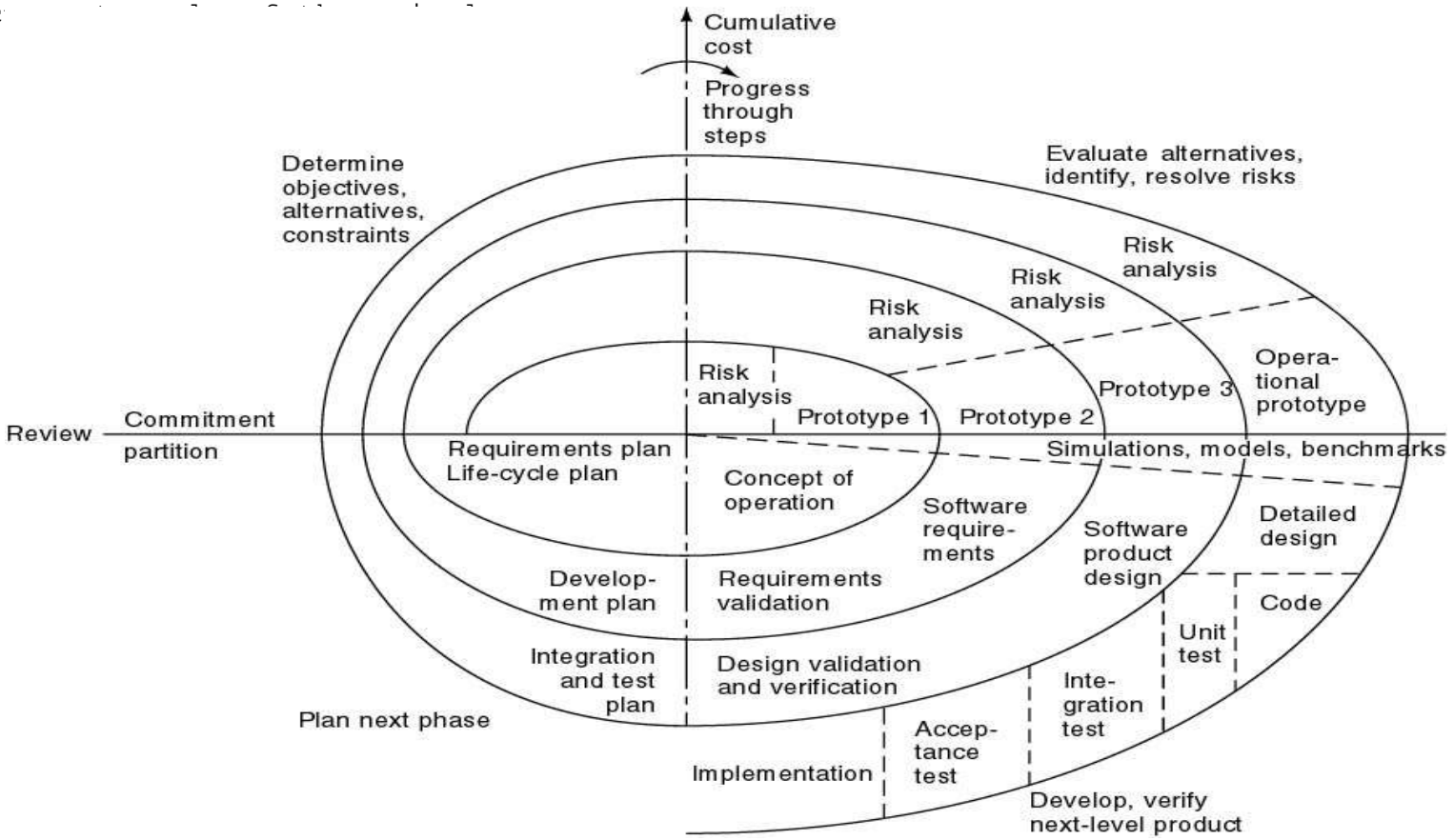
In Short:

- **Identify** the risk.
- **Solve** the risk.
- **If you can't solve it?** Shut it down.

Full Spiral Model

The **Full Spiral Model** is a way of managing projects by looking at how much you've spent and how far you've come as you circle around a plan. It uses two main measurements to track this:

- **Radial dimension:** This represents the total cost spent on the project so far.
- **Angular dimension:** This represents how much progress has been made through the c



Building a New Smartphone

- Imagine a tech company developing a revolutionary foldable phone.

1. The Cost (Radial Dimension)

- As the project "spirals" out, the distance from the center gets larger. This shows the **total money spent.**
- **Loop 1:** Researching screens (Cheap).
- **Loop 2:** Building a prototype (More expensive).
- **Loop 3:** Mass manufacturing (Very expensive).

2. The Progress (Angular Dimension)

- Each full circle represents a phase of the project. The "angle" shows **where you are** in that specific phase.
- If you are halfway through the circle, you might be done with the design but haven't started testing yet.

3. The "Kill Switch"

- If, during Loop 2, the team finds that the folding screen cracks every time (a **risk that cannot be resolved**), they **terminate the project immediately.** This prevents them from moving to the "expensive" Loop 3 and losing more money.

Analysis of Spiral Model

Strengths and Weaknesses

- Based on your notes, here is the breakdown of why teams use this model and where it falls short:
- **Strengths**
- **Easy to judge how much to test:** Because each loop ends with an evaluation, it's clear how much testing is needed before moving to the next level.
- **No distinction between development and maintenance:** The model treats "fixing" things and "building" new things as part of the same continuous loop.
- **Weaknesses**
- **For large-scale software only:** It is too "heavy" and expensive for small projects.
- **For internal (in-house) software only:** It works best when the developers and the "client" are in the same company, as external contracts often require more rigid timelines than a spiral allows.

An Airline Booking System

Imagine a major airline building a new internal booking system:

1. **Loop 1 (The Center):** They build a basic text-only version. Cost is low (Radial), progress is one full circle (Angular).
2. **Loop 2 (The Growth):** They add credit card processing. The spiral gets wider because they hired more people (Cost/Radial increase).
3. **The Risk Check:** During Loop 2, they realize the server can't handle 1,000 users. Because this is a **Full Spiral Model**, they don't just keep building; they focus the entire next loop on fixing that server risk before adding any new features.

Object-Oriented Life-Cycle Models

- These models are built for modern software where parts need to be reused and updated constantly. They focus on **iteration** (doing things over to improve them) both within and between different phases of work.

Common Models in this Category:

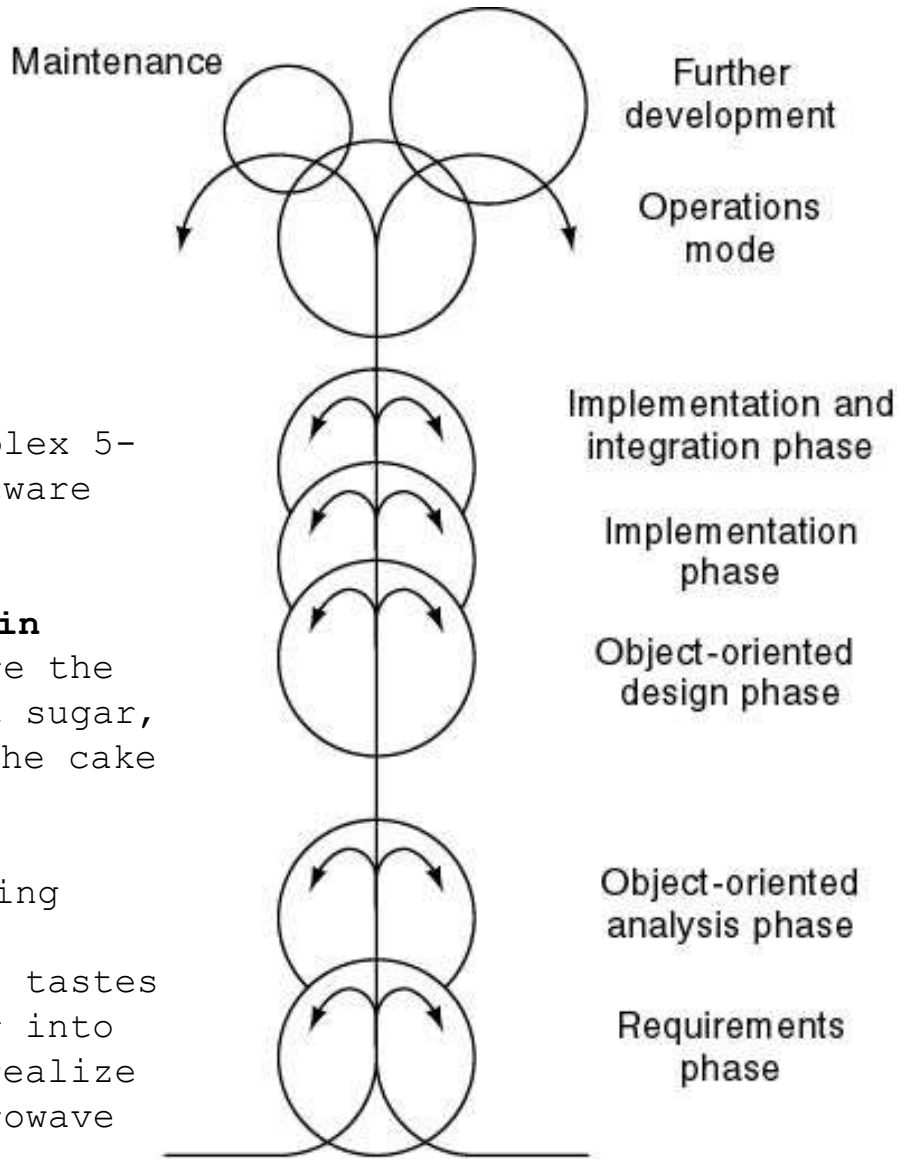
- **Fountain Model:** Phases overlap like water in a fountain.
- **Recursive/Parallel Life Cycle:** Different parts of the project are built at the same time.
- **Unified Process:** A popular industry standard for incremental growth.
- **The Big Idea**
- All of these models rely on three pillars: **Iteration**, **Parallelism**, and **Incremental development.**

The Main Danger: CABTAB

- The biggest risk with these flexible models is **CABTAB(C**ode **A**nd **B**uild, **T**est **A**nd **B**ug) —an **undisciplined approach** to software development.
- This happens when teams get too messy with iterations and lose track of the original plan.
- **The CABTAB approach**: Instead of following a structured plan, the developers just start coding randomly. They change the database, then realize it breaks the login, then jump to fixing the login without finishing the Stories feature, and eventually lose track of what has been tested or completed.

Fountain Model

- Circles (phases)
  Overlap (parallelism)
- Arrows (iteration)
- Smaller maintenance circle



"The Messy Kitchen"

Imagine you are trying to bake a complex 5-tier wedding cake (a large-scale software project).

•**A Disciplined Model (like the Fountain Model):** You have a recipe, you prepare the batter, and if you realize you forgot sugar, you go back a step to add it before the cake goes in the oven.

•**The CABTAB Approach:** You start throwing flour and eggs into a bowl without measuring. You put it in the oven, it tastes bad (Bug), so you try to inject sugar into the baked cake (Patching). Then you realize the middle is raw, so you try to microwave just the center.

By the end, you have a "cake," but it's a

Here is the simplest way to see the difference using a **Restaurant Kitchen** as an example.

- **The Problem: A customer wants to change their order from "Pasta" to "Gluten-Free Pasta."**

1. The Fountain Model (Disciplined Iteration)

- In this model, the kitchen is flexible but follows a clear structure.
- **The "Flow Back":** The chef stops and goes back to the "Ingredients" stage (Analysis/Design) to check if they have gluten-free flour.
- **The Overlap:** While the dough is being made (Implementation), the chef starts preparing the sauce (Parallelism) because they know it works for both.
- **The Result:** A high-quality meal that safely meets the new requirement.

CABTAB (Undisciplined Chaos)

- **CABTAB** stands for an undisciplined approach where the team just "hacks" things together.
- **The "Code & Build":** The cook tries to wash the sauce off the regular pasta to "make it" gluten-free.
- **The "Test & Bug":** The customer gets sick (The Bug). The cook then tries to give them a stomach pill (The Patch).
- **The Danger:** There is no plan or discipline. The kitchen becomes a mess, and eventually, the restaurant has to close because the food is unsafe.

- In Object-Oriented Life-Cycle models, we use **Iteration** (repeating steps) and **Parallelism** (doing two things at once). CABTAB is the "evil twin" of these concepts:
- **Iteration vs. CABTAB:** Iteration is "Let's refine this." CABTAB is "I'll just keep changing it until it stops crashing."
- **Parallelism vs. CABTAB:** Parallelism is "Team A builds the engine while Team B builds the wheels." CABTAB is "Everyone is building everything at once and nothing fits together."

How to Spot CABTAB

- **No Documentation:** No one writes down how the code works.
- **Endless Bug Fixing:** You fix one bug, and three more appear because the foundation is weak.
- **"Just One More Change":** Developers are constantly "tweaking" things without testing how